Scenix

3D Scene Reconstruction using Gaussian Splatting

Internship at CAVE labs, PES University, RR Campus

Deeptha S , PES1UG24CS144

Duration: 1 June 2026 – 17 July 2026

Project Guides: Professor Dr Adithya Balasubramanyam and Manoj Kumar HR

Repository: https://github.com/deep-1005/Scenix

Project Overview

Scenix is an end-to-end platform that converts raw 360° captures of any scene — rooms, buildings, labs, facilities — into a navigable, measurable, photorealistic 3D Gaussian splat with evidence (object) detection and classification as future scope. A scene is photographed with an Insta360 camera; each equirectangular panorama is sliced into 15 pinhole perspective views (5 yaw × 3 pitch, 90° FOV, 1400×1400) with py360convert's e2p; camera poses and a sparse point cloud are recovered with COLMAP (exhaustive matching, locked PINHOLE intrinsics for the synthetic views); the sparse cloud is statistically cleaned with auto-tuned RANSAC and DBSCAN and the cleaned cloud is promoted into sparse/0 so training provably uses it; a photorealistic Gaussian splat is trained with FastGS under a tuned densification preset; the trained splat is de-noised with 3dgsconverter; and the result is served through a FastAPI + Celery + React platform with per-stage progress, resumable jobs and an in-browser splat viewer

The work was divided among three team members, each owning one workstream end-to-end while collaborating on integration and testing. This report covers my individual contribution: the core reconstruction stages — COLMAP structure-from-motion, FastGS Gaussian splat training, point-cloud and splat cleaning, and the reconstruction quality work that set the platform's training defaults.

My Role and Responsibilities

I owned the core reconstruction stages of the pipeline: COLMAP structure-from-motion (camera pose estimation and sparse point clouds), FastGS Gaussian splat training, inspection and cleanup of the resulting models in SuperSplat and MeshLab, and statistical point-cloud cleaning with RANSAC and DBSCAN (auto-tuned, with the cleaned cloud promoted into the training input). My goal was to turn the preprocessed image sets into accurate, high-quality 3D reconstructions and to establish the training settings that the automated platform now uses by default in its Celery pipeline.

Work Carried Out

Foundations, COLMAP baseline and tool evaluation (1–8 June)

The first week covered theory and research (3D data science, photogrammetry, Gaussian splatting), team formation, and environment setup (Blender, Unity). I installed COLMAP and validated it on the official South Building dataset, viewing the sparse reconstruction and the ring of registered camera poses in the COLMAP GUI and MeshLab, then moved to our own captures: a bottle photographed on a phone and a book captured with the 360° camera.

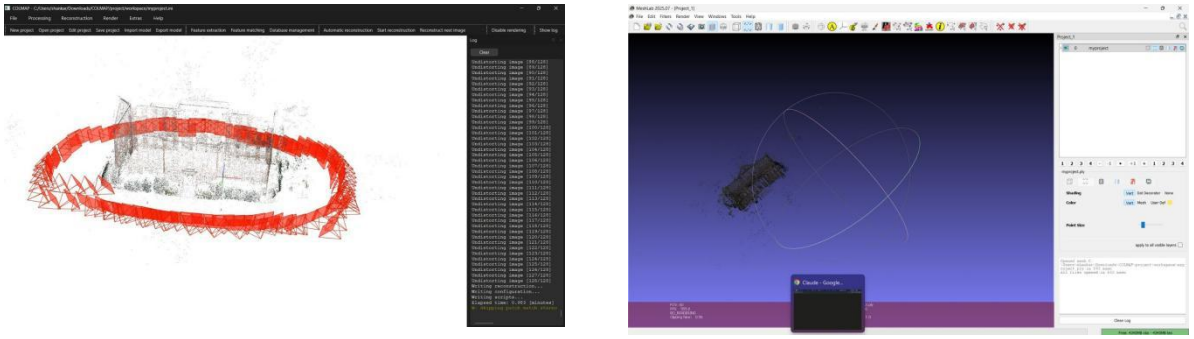


Figure 1 — COLMAP baseline on the South Building dataset: registered camera ring in the COLMAP GUI (left) and the sparse model inspected in MeshLab (right).

Figure 2 — First test captures — a bottle photographed on a phone and a book captured with the 360° camera.

I then benchmarked alternative reconstruction tools on the same book capture. COLMAP produced the cleanest camera registration; RealityScan was faster and could export an OBJ file that we loaded into a Unity test scene, but proved unusable for inside-out room scans; Brush trained a Gaussian splat directly (30,000 steps on an RTX 4090, ~394k splats); Polycam and LumaAI produce textured meshes from 2D photos rather than splats; PostShot outputs a proprietary .psht instead of .ply; and Metashape, Meshroom and Nerfstudio were blocked by GPU and storage constraints. This fixed the open COLMAP + FastGS chain as the only route meeting our quality, format and automation requirements.

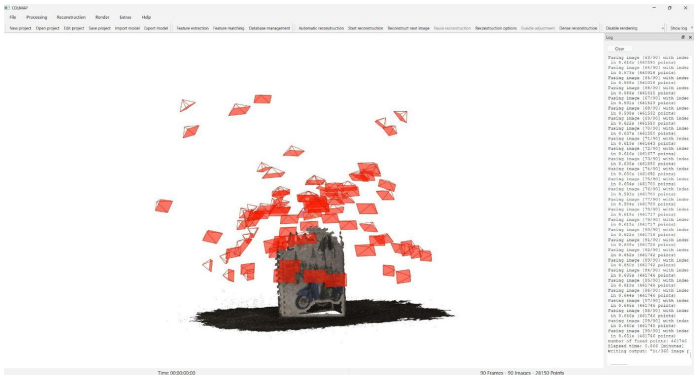


Figure 3 — Tool-evaluation phase: COLMAP reconstruction of the book capture with densely registered camera poses (red).

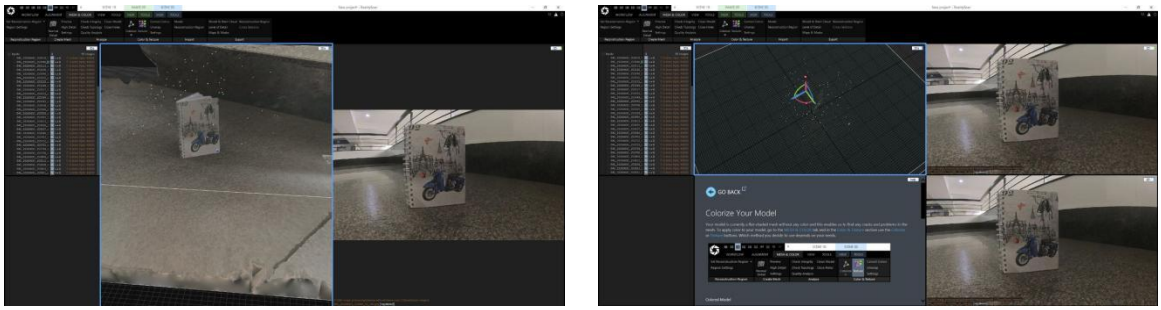


Figure 4 — RealityScan benchmarking on the book — image alignment (left) and the model colorization step (right).

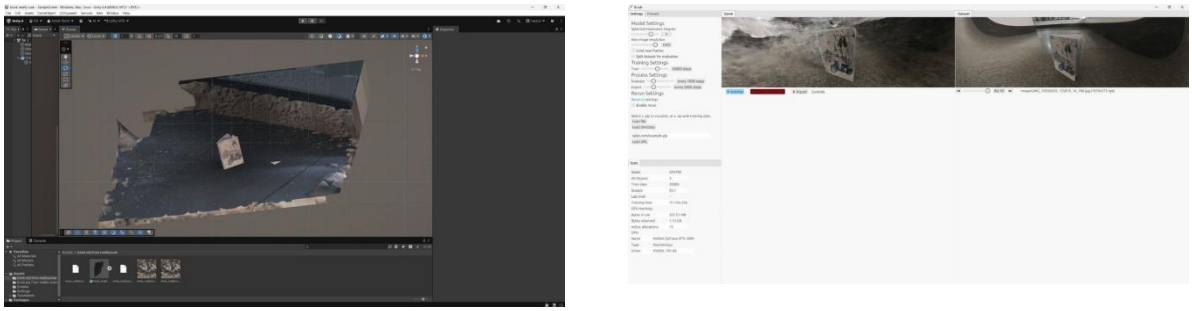


Figure 5 — RealityScan's OBJ imported into a Unity test scene (left); Brush training a Gaussian splat of the book — 30,000 steps on an RTX 4090 (right).

An early blocking difficulty was input format: equirectangular 360° images fed directly to COLMAP or RealityScan simply fail, and GUI converters were dead ends (Pano2VR caps its free tier at four photos). We moved to scripted equirectangular-to-perspective conversion in Python, which unblocked my COLMAP runs and later became Stage 2 of the platform — now generating 15 pinhole views per panorama (5 yaw angles × 3 pitch angles at 90° FOV), which gives COLMAP far better inter-view overlap than plain 6-face cubemaps.

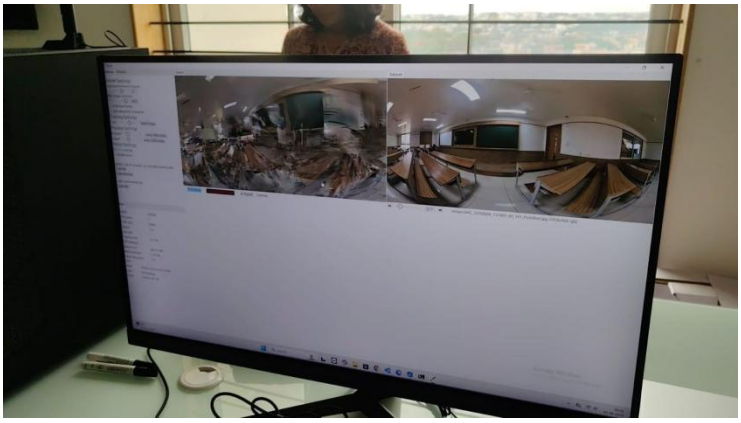


Figure 6 — The input-format problem: raw equirectangular Insta360 exports — a format COLMAP cannot consume directly.

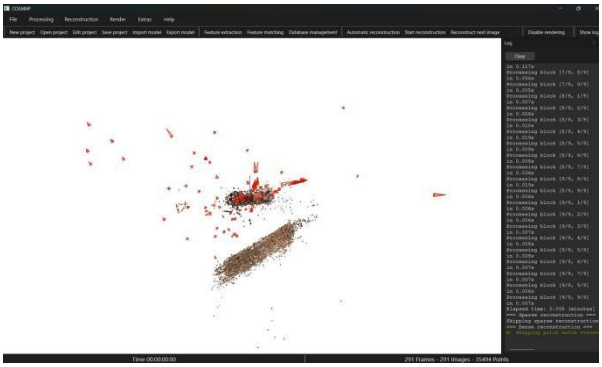
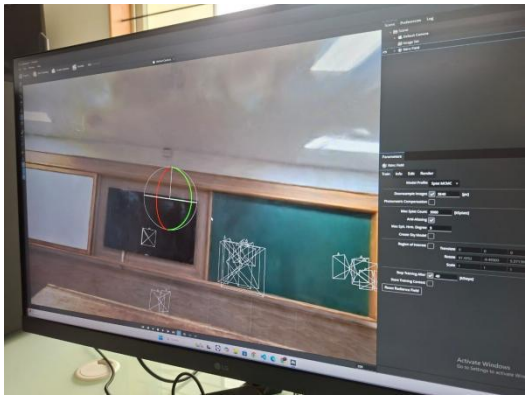


Figure 7 — Early failure mode before proper view generation: a scattered, unusable sparse model from naive inputs.

Scaling reconstruction (9–16 June)

I scaled COLMAP to large image sets and moved it from the GUI to the terminal to prepare for automation. A critical finding was that unordered generated views require exhaustive matching: consecutive filenames are not spatially adjacent, so sequential matching finds far too few correspondences. Attempts at 4200 and 5040 images failed on both lab systems and our laptop purely for lack of disk space, and cloud fallbacks (Nerfstudio and Splatware on Google Colab) also did not work; after we were allotted a new lab system I standardised on 1050 views (15 × 70 panoramas) as the largest reliable working set. The 1050-view run produced a dense, well-registered sparse model of the classroom, clearly superior to the 444-view run.



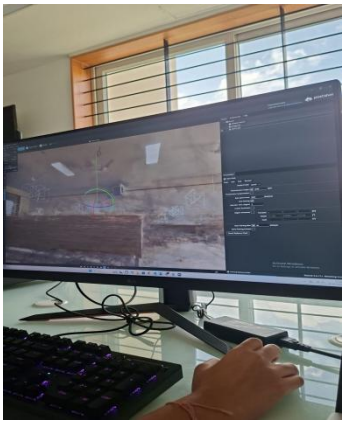


Figure 8 — Early classroom reconstruction experiments during the FastGS/SOG/PlayCanvas evaluation week.

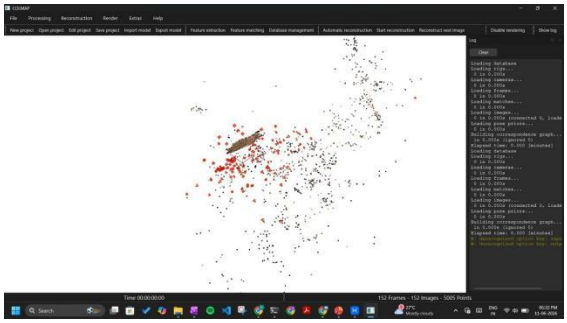


Figure 9 — Scaling work: a sparse, noisy 400-image attempt (left) and COLMAP driven from the terminal in preparation for automation (right).

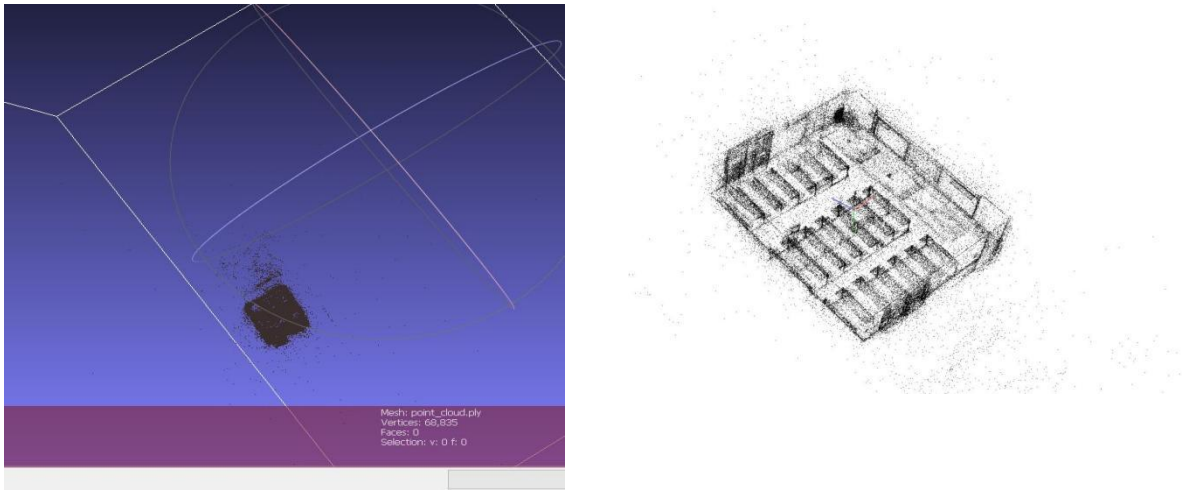


Figure 11 — Classroom point cloud (point_cloud.ply) in MeshLab (~68.8k vertices, left) and the top-down view with rows of desks clearly resolved (right).

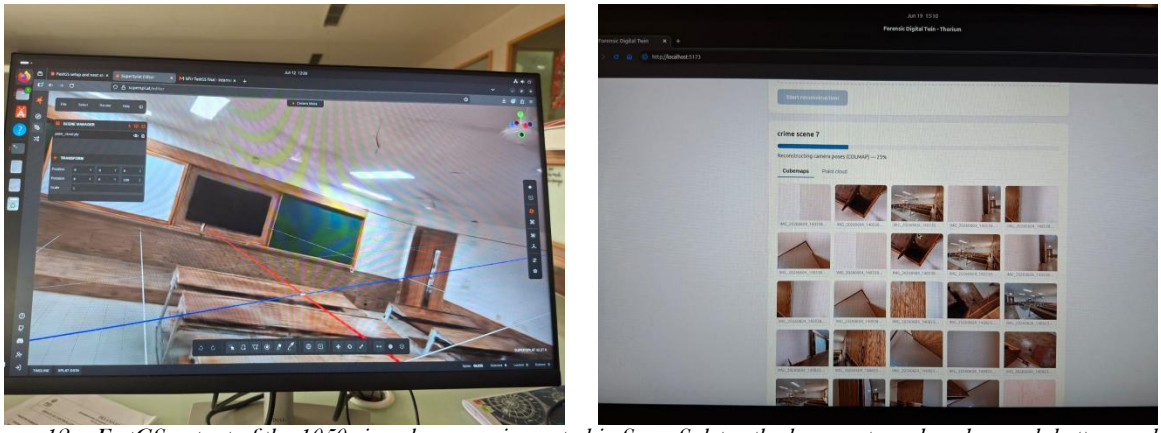


Figure 12 — FastGS output of the 1050-view classroom inspected in SuperSplat — the larger set produced a much better model than the 444-view run.

FastGS training and platform integration (9–30 June)

I set up FastGS and debugged its environment, including the ModuleNotFoundError for diff_gaussian_rasterization_fastgs that initially blocked all training. Two settings emerged as decisive: training beyond roughly 30,000 iterations overfits the training views rather than improving them, and FastGS silently downscales any input wider than 1600 px unless the -r flag is set explicitly — so input resolution must always be controlled deliberately. The same missing-rasterizer failure resurfaced when FastGS was wired into the web platform (training jobs exited with code 1 while the COLMAP stage completed normally) and was overcome by rebuilding the CUDA extension inside the conda environment and injecting CUDA_HOME/PATH/LD_LIBRARY_PATH into the worker's subprocess environment in tasks.py.

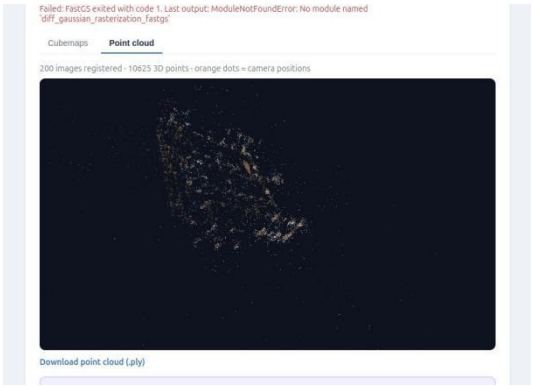


Figure 13 — Integration difficulty: FastGS exiting with the missing diff_gaussian_rasterization_fastgs rasterizer inside the platform while the COLMAP point-cloud stage (200 images registered) completed normally.

I implemented RANSAC and DBSCAN cleaning of the COLMAP point clouds with statistical outlier removal, using Optuna and an auto-tuning routine (parameters derived from the cloud's average point spacing) so no manual tuning is needed per scene. A key engineering detail: FastGS reads sparse/0/points3D.ply directly if it exists, so the cleaning stage must overwrite that exact file — the pipeline now promotes the cleaned cloud into place, guaranteeing training runs on cleaned points rather than silently using the raw cloud.

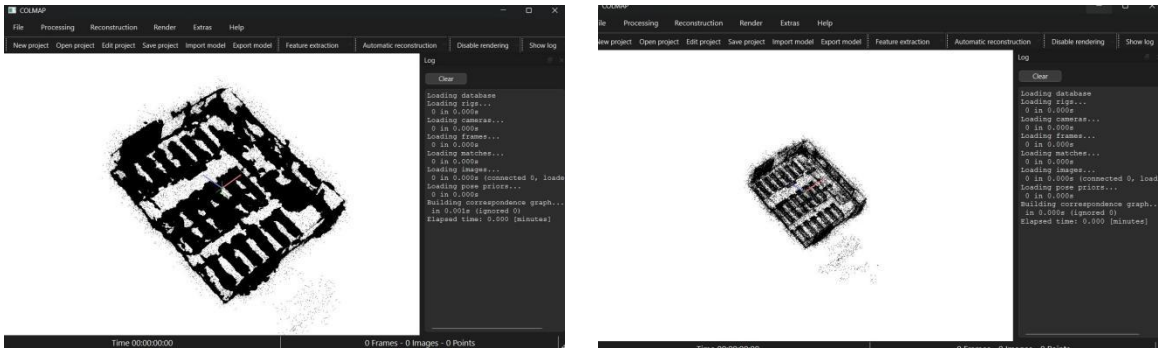


Figure 14 — RANSAC/DBSCAN cleaning outputs on the classroom cloud — mesh-file view (left) and the cleaned cloud, closely matching the raw COLMAP result once parameters are tuned (right).

By the end of June FastGS was integrated properly into the platform: the full chain ran as one tracked Celery job with per-stage progress, and a 420-view run reconstructed correctly through the web UI. Each stage's completion is verified on disk, which is what makes the platform's resume feature safe.

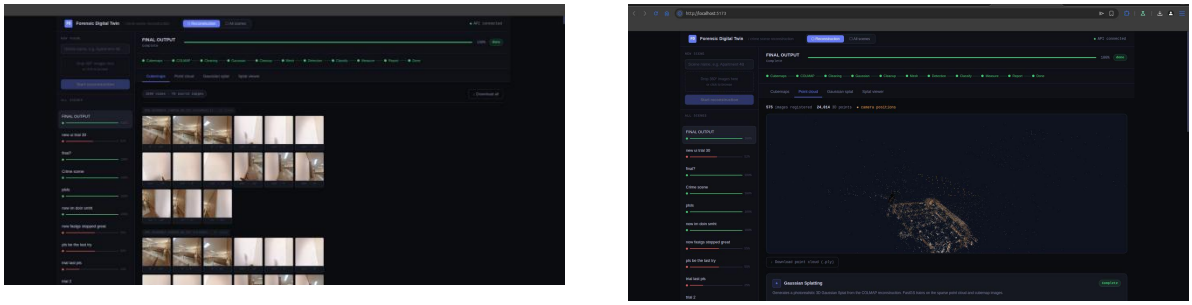


Figure 15 — The platform's reconstruction tabs during integration: generated perspective views (left) and the sparse point cloud with camera positions (right).

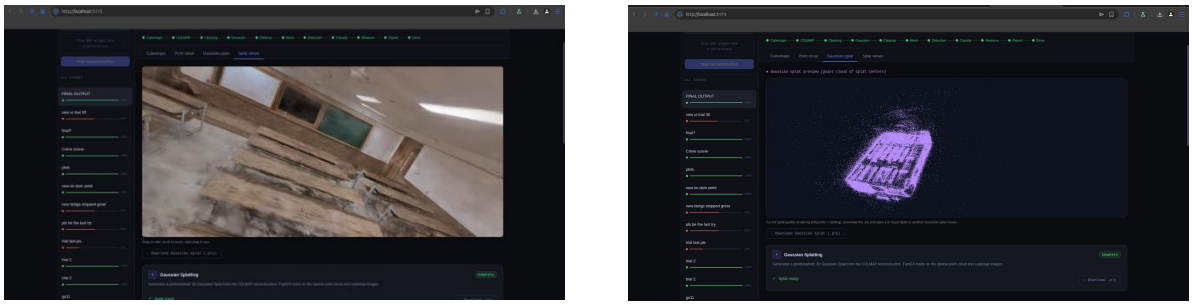


Figure 16 — Gaussian-splat centers preview (left) and the in-browser splat viewer (right) after FastGS integration.

Figure 17 — Classroom Gaussian splat in the platform's viewer after the 420-view integration run — benches, boards and floor rendered photorealistically.

Splat quality work and remote access (1–13 July)

The final tuning pass produced the training preset the platform now ships in tasks.py: 30,000 iterations with saves at 15,000 and 30,000, a denser densification schedule (interval 100, densify until iteration 15,000, lowered gradient thresholds), lambda_dssim raised to 0.3 for sharper structure, and PSNR/SSIM/LPIPS evaluated at test iterations for objective comparison across runs. Custom opacity/scale regularization flags were trialled but the FastGS build rejects them, so floater and needle suppression is handled instead by the cleaned-cloud promotion before training and a post-training 3dgsconverter pass (opacity threshold + statistical outlier removal) on the trained splat. I also evaluated Self-Organizing Gaussians (SOG), concluding it cannot replace FastGS in our pipeline, and used Blender for evaluation of trained models. Validation extended outdoors: the south-eastern campus building reconstructed and rendered cleanly in the platform's splat viewer. In parallel we fought remote-demo problems — LAN/nginx hosting would not run and ngrok exhausted its free-tier bandwidth streaming splats — solved with SSH tunnelling, with a duckdns subdomain identified for future demos.

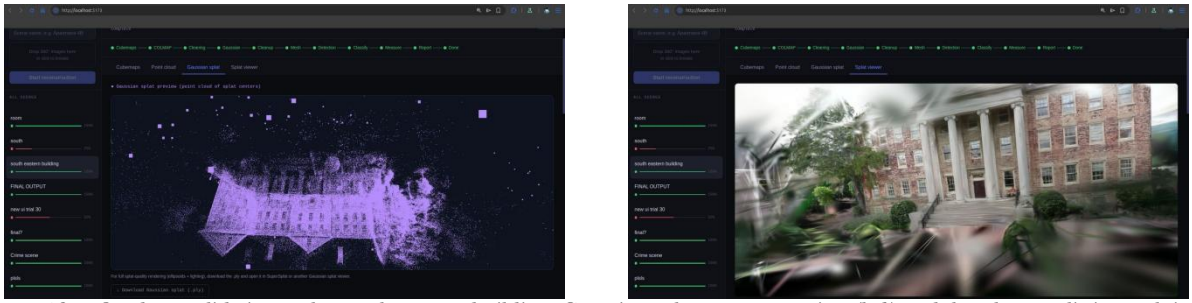


Figure 18 — Outdoor validation — the south-eastern building: Gaussian-splat centers preview (left) and the photorealistic result in the in-browser splat viewer (right).

SuGaR trial, evidence layer and external validation (14–17 July)

In the final days I attempted SuGaR integration for surface-aligned meshing: it conflicted with our FastGS build (training errors) and the meshes it did produce were not good enough to justify the added complexity, so SuGaR was removed from the active pipeline and meshing was kept on the roadmap through the splat-for-viewing / mesh-for-measurement dual-output design.

Conclusion

To confirm the reconstruction chain generalises beyond our own captures, I validated it end-to-end on public Mendeley datasets — a faculty-of-arts building and a parking lot — both of which reconstructed well through the platform with no dataset-specific tuning. This work also explained an apparent image-loss bug: the platform registered 69 of 80 uploaded images because near-duplicate and low-parallax frames contribute nothing to registration and are dropped, and oversized uploads are evenly subsampled to the panorama cap.

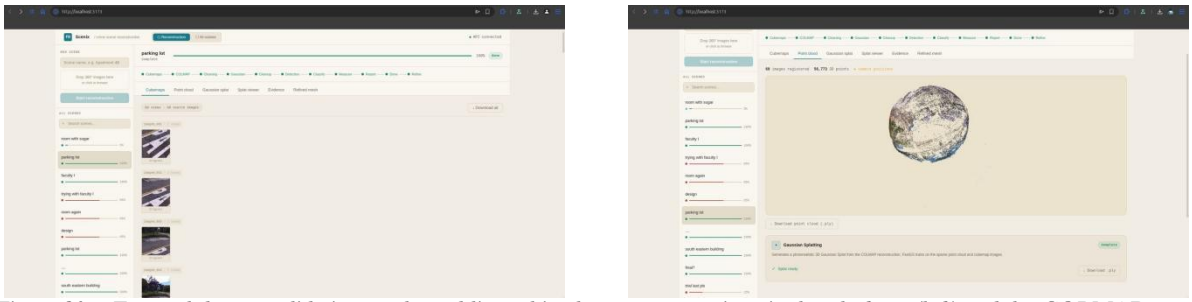


Figure 20 — External-dataset validation on the public parking-lot set: source views in the platform (left) and the COLMAP sparse point cloud, 66 images registered (right).

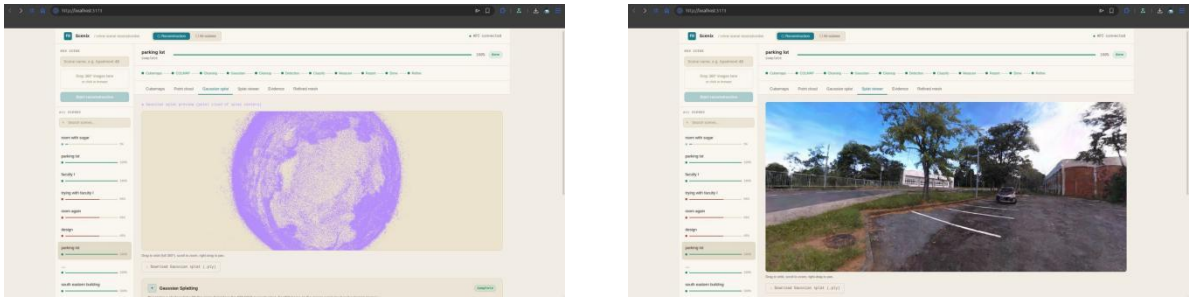


Figure 21 — Parking-lot Gaussian splat: splat-centers preview (left) and the final photorealistic reconstruction explored in the in-browser splat viewer (right).

Over the course of this internship, I took the core reconstruction pipeline of Scenix from a set of manual, GUI-driven experiments into a reliable, automated stage of a production platform. Working through the practical failure modes of COLMAP and FastGS — from format mismatches and rasterizer build errors to storage limits and silent resolution downscaling — taught me that robust 3D reconstruction is less about any single algorithm and more about disciplined engineering around it: verifying every stage on disk, controlling inputs deliberately, and measuring quality objectively rather than by eye. The training preset and cleaning pipeline I established are now the defaults the platform ships with, and the classroom and campus-building reconstructions stand as evidence that the approach generalises beyond a single test case. Beyond the technical outcome, this project gave me a much deeper appreciation for how photogrammetry, neural rendering, and systems engineering have to work together for a research idea to become something usable end to end.

Key Outcomes

- Reliable COLMAP reconstruction of full rooms from generated perspective views (15 per panorama via e2p), scaled to 1050+ images, with exhaustive matching established as mandatory for unordered views and locked PINHOLE intrinsics used for the synthetic crops.
- Working FastGS training with a tuned, shipped preset: 30k iterations, tuned densification schedule (interval 100, until 15k, lowered gradient thresholds), lambda_dssim 0.3, mid-training saves, and PSNR/SSIM/LPIPS test evaluations — producing clear photorealistic splats of the classroom and outdoor scenes.
- Automated RANSAC + DBSCAN + SOR point-cloud cleaning with auto-tuned parameters, and cleaned-cloud promotion into sparse/0/points3D.ply so FastGS provably trains on cleaned points.
- Post-training splat cleanup via 3dgsconverter (opacity threshold + statistical outlier removal), replacing the unsupported in-training regularizer flags.
- End-to-end validation on external public datasets (Mendeley faculty-of-arts and parking-lot image sets), confirming the reconstruction chain generalises beyond our own captures.
- Systematic tool benchmarking (RealityScan, Brush, Polycam/LumaAI, PostShot, Metashape, Meshroom) and the SOG-vs-FastGS comparison that fixed COLMAP + FastGS as the pipeline core.

Challenges and how it was overcome

- Equirectangular 360° images fed directly to COLMAP and RealityScan failed outright, and GUI converters capped out at four photos on the free tier — overcome by scripting equirectangular-to-perspective conversion in Python (py360convert e2p), which became Stage 2 of the platform; switching from 6 cubemap faces to 15 overlapping perspective views per panorama materially improved registration.
- Sequential matching found far too few correspondences on unordered generated views — overcome by switching to exhaustive matching, accepting the extra compute as the price of correct registration.
- Storage limits killed the 4200- and 5040-image COLMAP runs on both lab systems and our laptop, and Colab fallbacks failed — overcome by moving to a newly allotted lab system and standardising on the 1050-view working set.
- FastGS was blocked by the missing diff_gaussian_rasterization_fastgs rasterizer, first locally and again inside the platform — overcome by rebuilding the CUDA extension in the conda environment and injecting CUDA_HOME/PATH/LD_LIBRARY_PATH into the Celery worker's FastGS subprocess.
- Splat quality plateaued and then degraded with more training: beyond ~30k iterations FastGS overfits — overcome by capping iterations and verifying quality with PSNR/SSIM/LPIPS at fixed test iterations rather than by eye.
- FastGS silently downscales inputs wider than 1600 px — overcome by controlling resolution deliberately end-to-end: views are generated at a fixed 1400×1400 and the -r flag is always set explicitly in the preset.
- SuGaR meshing conflicted with our FastGS build and produced weak meshes — overcome by removing it from the active pipeline and keeping meshing on the roadmap via the splat-for-viewing / mesh-for-measurement dual-output design.
- Remote demos kept failing — LAN/nginx hosting would not run and ngrok exhausted its free-tier bandwidth streaming splats — overcome with SSH tunnelling for remote access, with a duckdns subdomain identified for future demos.

Tools & Technologies Used

- COLMAP 3.13 (GUI and terminal, conda-forge CUDA build), pycolmap, MeshLab
- FastGS (Gaussian splat training), 3dgsconverter (splat cleanup), SuperSplat (inspection)
- py360convert (e2p perspective-view generation), OpenCV, Pillow/pillow-heif
- Open3D, RANSAC, DBSCAN, statistical outlier removal, Optuna (point-cloud cleaning and tuning)
- PSNR / SSIM / LPIPS quality metrics, Blender (evaluation)
- FastAPI, Celery, Redis, SQLAlchemy (pipeline integration context)
- Self-Organizing Gaussians and SuGaR (evaluated), Nerfstudio (explored)
- RealityScan, Brush, Polycam/LumaAI, PostShot (tool benchmarking); Unity and Blender (early mesh tests)

Future Scope

- Scenix is functional end-to-end today, but several parts of the pipeline are deliberately first-pass implementations with clear room to grow. This section lays out what's planned next and why it matters, for anyone evaluating where the project is headed.

- Richer, more reliable evidence (object) detection — Detection currently runs on a fixed set of text prompts through OwlViT. Planned work: a configurable prompt set so a user can tell the system what kinds of objects to look for per scene, and an upgrade path to stronger open-vocabulary detectors (Grounding DINO / SAM 2) for higher recall and tighter boxes. Each detected object would also carry a supporting-view count — how many camera angles independently confirmed it — as a built-in confidence signal. This is the core research contribution of the project and the area with the most active development.

- A fuller evidence report— The report currently outputs a v1 PDF with room dimensions and a flat object table. The next version would attach a photo crop and supporting-view thumbnails to every object, include capture metadata (date, camera, number of panoramas, marker used), embed a render of the reconstructed scene, and offer a DOCX export alongside the PDF. The goal is a report that can stand on its own as a shareable summary of a scene, not just a table.

- More flexible scale calibration — Measurements currently rely on a single ArUco marker being visible in at least two views. Future versions would support multiple markers for cross-validation, and optionally a LiDAR reference for scenes where placing a printed marker isn't practical.

- GPU workers that scale — Right now the Celery worker runs on whichever machine it's launched on, with CUDA visibility handled by manually injected environment variables. The production target is Docker containers orchestrated by Kubernetes with the NVIDIA container runtime, so multiple reconstruction jobs can run in parallel across a GPU cluster instead of one machine.

Acknowledgment

I would like to express my gratitude to my guides **Professor Dr Adithya Balasubramanyam and Manoj Kumar HR, CAVE labs, PESU** for their continuous guidance, assistance and encouragement throughout the development of this project.

I am grateful to the internship coordinator Prof Mahitha G and Dr Bhargavi M, Dept. of Computer Science and Engineering , PES University for organizing, managing and helping out with the entire process.

I take this opportunity to thank Dr. Shylaja S S, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof Jawahar Doreswamy, Chancellor, PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing me various opportunities and enlightenment every step of the way.

Finally, this internship could not have been completed without the continual support and encouragement I have received from my parents, my friends and my team.